

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

**«Запросы на выборку и модификацию данных. Представления. Работа
с индексами»**

Обучающийся Сафонова Арина Дмитриевна

Факультет прикладной информатики

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
 2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов.**
 3. Изучить графическое представление запросов и просмотреть историю запросов.
 4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.
- Составить список всех 2-местных номеров отелей, с ценой менее 200 т.р., упорядочив данные в порядке уменьшения стоимости.

```
SELECT
```

```
    h.name_hotel AS "ОТЕЛЬ",
```

```
    r.number_room AS "Номер",
```

```
    tr.count_places AS "Количество мест",
```

```
    p.price_per_day AS "Цена за сутки"
```

```
FROM hotel.room r
```

```
JOIN hotel.hotel h
```

```
    ON r.id_hotel = h.id_hotel
```

```
JOIN hotel.type_room tr
```

```
    ON r.id_type_room = tr.id_type_room
```

```
JOIN hotel.price p
```

```
    ON tr.id_type_room = p.id_room_type
```

```
WHERE tr.count_places = 2
```

```
    AND p.price_per_day < 200000
```

```
ORDER BY p.price_per_day DESC;
```

Query Query History

↶ ↷ ✕

↗

```

1  SELECT
2      h.name_hotel AS "ОТЕЛЬ",
3      r.number_room AS "Номер",
4      tr.count_places AS "Количество мест",
5      p.price_per_day AS "Цена за сутки"
6  FROM hotel.room r
7  JOIN hotel.hotel h
8      ON r.id_hotel = h.id_hotel
9  JOIN hotel.type_room tr
10     ON r.id_type_room = tr.id_type_room
11  JOIN hotel.price p
12     ON tr.id_type_room = p.id_room_type
13  WHERE tr.count_places = 2
14         AND p.price_per_day < 2000000
15  ORDER BY p.price_per_day DESC;

```

Data Output Messages Notifications
↗

☰
📄
▼
📋
▼
🗑️
🗄️
⬇️
📈
SQL

Showing rows: 1 to 2
✎
🖨️
Page No: of 1

⏪
⏴
⏵
⏩

	ОТЕЛЬ character varying 🔒	Номер integer 🔒	Количество мест integer	Цена за сутки numeric (10,2)
1	Grand Hotel	102	2	5000.00
2	Nevsky Palace	202	2	5000.00

```
SELECT

    id_order AS "Запись",

    id_guest AS "Постоялец",

    check_in_date AS "Дата заезда",

    check_out_date AS "Дата выезда"

FROM hotel.booking_order

WHERE check_out_date >= CURRENT_DATE - 14

AND check_out_date <= CURRENT_DATE;
```

Query
Query History

F

```

1  SELECT
2      id_order AS "Запись",
3      id_guest AS "Постоялец",
4      check_in_date AS "Дата заезда",
5      check_out_date AS "Дата выезда"
6  FROM hotel.booking_order
7  WHERE check_out_date >= CURRENT_DATE - 14
8      AND check_out_date <= CURRENT_DATE;

```

Data Output
Messages
Notifications

SQL

Showing rows: 1 to 1
Page No: 1 of 1

	Запись integer	Постоялец integer	Дата заезда date	Дата выезда date
1	1	1	2026-05-22	2026-05-24

- Чему равен общий суточный доход каждого отеля за последний месяц?

```

SELECT

    h.name_hotel AS "Отель",

    b.check_out_date AS "Дата",

    SUM(b.amount) AS "Суточный доход"

FROM hotel.booking_order b

JOIN hotel.room r

    ON b.id_room = r.id_room

JOIN hotel.hotel h

    ON r.id_hotel = h.id_hotel

WHERE b.check_out_date >= CURRENT_DATE - 30

AND b.check_out_date <= CURRENT_DATE

GROUP BY h.name_hotel, b.check_out_date;

```

Query

Query History

F

1

2

3

4

5

6

7

8

9

10

11

12

SELECT

h.name_hotel AS "Отель",

b.check_out_date AS "Дата",

SUM(b.amount) AS "Суточный доход"

FROM hotel.booking_order b

JOIN hotel.room r

ON b.id_room = r.id_room

JOIN hotel.hotel h

ON r.id_hotel = h.id_hotel

WHERE b.check_out_date >= CURRENT_DATE - 30

AND b.check_out_date <= CURRENT_DATE

GROUP BY h.name_hotel, b.check_out_date;

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

	Отель character varying	Дата date	Суточный доход numeric
1	Grand Hotel	2026-05-...	10000
2	Nevsky Palace	2026-05-...	8000

```
1 SELECT
2     h.name_hotel AS "Отель",
3     b.check_out_date AS "Дата",
4     SUM(b.amount) AS "Суточный доход"
5 FROM hotel.booking_order b
6 JOIN hotel.room r
7     ON b.id_room = r.id_room
8 JOIN hotel.hotel h
9     ON r.id_hotel = h.id_hotel
10 WHERE b.check_out_date >= CURRENT_DATE - 30
11     AND b.check_out_date <= CURRENT_DATE
12 GROUP BY h.name_hotel, b.check_out_date;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

Showing rows: 1 to 2   Page No: 1 of 1    

	ОТЕЛЬ character varying	ДАТА date	СУТОЧНЫЙ ДОХОД numeric
1	Grand Hotel	2026-05-...	10000
2	Nevsky Palace	2026-05-...	8000

- Составить список свободных номеров заданного отеля на текущий день.

SELECT

h.name_hotel AS "Отель",

```
r.number_room AS "Homep",
```

r.floor AS "Этаж",

r.status_room AS "Статус"

FROM hotel.room r

JOIN hotel.hotel.hk

ON r.id_hotel = h.id_hotel

```
WHERE h.name_hotel = 'Grand Hotel'
```

```
AND r.status_room = 'free';
```

Query Query History

```

1  SELECT
2      h.name_hotel AS "Отель",
3      r.number_room AS "Номер",
4      r.floor AS "Этаж",
5      r.status_room AS "Статус"
6  FROM hotel.room r
7  JOIN hotel.hotel h
8      ON r.id_hotel = h.id_hotel
9  WHERE h.name_hotel = 'Grand Hotel'
10     AND r.status_room = 'free';

```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1

	Отель character varying	Номер integer	Этаж integer	Статус character varying
1	Grand Hotel	101	1	free

- Найти общие потери от незанятых номеров за текущий день по всей сети.

```

SELECT
    SUM(p.price_per_day) AS "Общие потери"
FROM hotel.room r
JOIN hotel.type_room tr
    ON r.id_type_room = tr.id_type_room
JOIN hotel.price p
    ON tr.id_type_room = p.id_room_type
WHERE r.status_room = 'free';

```


Query Query History

```

1  SELECT
2      h.name_hotel AS "Отель",
3      COUNT(r.id_room) AS "Количество незанятых номеров"
4  FROM hotel.hotel h
5  JOIN hotel.room r
6  ON r.id_hotel = h.id_hotel
7  WHERE r.status_room = 'free'
8  GROUP BY h.name_hotel
9  HAVING COUNT(r.id_room) = (
10     SELECT COUNT(r2.id_room)
11     FROM hotel.room r2
12     WHERE r2.status_room = 'free'
13     GROUP BY r2.id_hotel
14     ORDER BY COUNT(r2.id_room) DESC
15     LIMIT 1
16 )

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	Отель character varying	Количество незанятых номеров bigint
1	Grand Hotel	1
2	Nevsky Palace	1

- Определить самый популярный тип номеров за последний год.

```

SELECT
    tr.name_type_room AS "Тип номера",
    COUNT(b.id_order) AS "Количество бронирований"
FROM hotel.booking_order b
JOIN hotel.room r
ON b.id_room = r.id_room
JOIN hotel.type_room tr
ON r.id_type_room = tr.id_type_room
WHERE b.check_in_date >= CURRENT_DATE - 365
GROUP BY tr.id_type_room, tr.name_type_room
HAVING COUNT(b.id_order) = (
    SELECT COUNT(b2.id_order)
    FROM hotel.booking_order b2
    JOIN hotel.room r2
    ON b2.id_room = r2.id_room
    WHERE b2.check_in_date >= CURRENT_DATE - 365
    GROUP BY r2.id_type_room
    ORDER BY COUNT(b2.id_order) DESC
    LIMIT 1
)

```


Query
Query History

```

1  SELECT
2      tr.name_type_room AS "Тип номера",
3      COUNT(b.id_order) AS "Количество бронирований"
4  FROM hotel.booking_order b
5  JOIN hotel.room r
6  ON b.id_room = r.id_room
7  JOIN hotel.type_room tr
8  ON r.id_type_room = tr.id_type_room
9  WHERE b.check_in_date >= CURRENT_DATE - 365
10 GROUP BY tr.id_type_room, tr.name_type_room
11 HAVING COUNT(b.id_order) = (
12     SELECT COUNT(b2.id_order)
13     FROM hotel.booking_order b2
14     JOIN hotel.room r2
15     ON b2.id_room = r2.id_room
16     WHERE b2.check_in_date >= CURRENT_DATE - 365
17     GROUP BY r2.id_type_room
18     ORDER BY COUNT(b2.id_order) DESC
19     LIMIT 1
20 )

```

Data Output
Messages
Notifications

Showing rows: 1 to 1
Page No: 1 of 1

	Тип номера character varying (100)	Количество бронирований bigint
1	Single	2

Задание 3. Создайте представления:

- Для турагентов (поиск свободных номеров в отелях).

```
CREATE VIEW hotel.view_free_rooms_for_agents AS
```

```
SELECT
```

```
h.name_hotel AS "Отель",
```

```
h.address AS "Адрес",
```

```
r.number_room AS "Номер",
```

```
r.floor AS "Этаж",
```

```
tr.name_type_room AS "Тип номера",
```

```
tr.count_places AS "Количество мест",
```

```
r.status_room AS "Статус",
```

```
p.price_per_day AS "Цена за сутки"
```

```
FROM hotel.room r
```

```
JOIN hotel.hotel h
```

```
ON r.id_hotel = h.id_hotel
```

```
JOIN hotel.type_room tr
```

```
ON r.id_type_room = tr.id_type_room
```

```
JOIN hotel.price p
```

ON tr.id_type_room = p.id_room_type

WHERE r.status_room = 'free';

Query Query History

```
1 CREATE VIEW hotel.view_free_rooms_for_agents AS
2 SELECT
3     h.name_hotel AS "Отель",
4     h.address AS "Адрес",
5     r.number_room AS "Номер",
6     r.floor AS "Этаж",
7     tr.name_type_room AS "Тип номера",
8     tr.count_places AS "Количество мест",
9     r.status_room AS "Статус",
10    p.price_per_day AS "Цена за сутки"
11 FROM hotel.room r
12 JOIN hotel.hotel h
13     ON r.id_hotel = h.id_hotel
14 JOIN hotel.type_room tr
15     ON r.id_type_room = tr.id_type_room
16 JOIN hotel.price p
17     ON tr.id_type_room = p.id_room_type
18 WHERE r.status_room = 'free';
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 57 msec.

Query Query History

```
1 SELECT *
2 FROM hotel.view_free_rooms_for_agents;
```

Data Output Messages Notifications

	Отель character varying	Адрес character varying	Номер integer	Этаж integer	Тип номера character varying (100)	Количество мест integer	Статус character varying	Цена за сутки numeric (10,2)
1	Nevsky Palace	Nevsky prospect ...	101	1	Single	1	free	3000.00
2	Grand Hotel	Tverskaya street ...	101	1	Single	1	free	3000.00

- Для владельца компании (информация о доходах каждого отеля в сети за прошедший месяц).

```
CREATE VIEW hotel.view_hotel_income_last_month AS
SELECT
    h.name_hotel AS "Отель",
    SUM(b.amount) AS "Доход за прошедший месяц"
FROM hotel.booking_order b
JOIN hotel.room r
    ON b.id_room = r.id_room
JOIN hotel.hotel h
    ON r.id_hotel = h.id_hotel
WHERE b.check_out_date >= CURRENT_DATE - 30
    AND b.check_out_date <= CURRENT_DATE
GROUP BY h.name_hotel
```

The screenshot shows a database IDE interface. At the top, there are tabs for "Query" and "Query History". The "Query" tab is active, displaying a SQL script. A tooltip with "Execute script" and "F5" is visible over the script. The script is as follows:

```
1 CREATE VIEW hotel.view_hotel_income_l AS
2 SELECT
3     h.name_hotel AS "Отель",
4     SUM(b.amount) AS "Доход за прошедший месяц"
5 FROM hotel.booking_order b
6 JOIN hotel.room r
7     ON b.id_room = r.id_room
8 JOIN hotel.hotel h
9     ON r.id_hotel = h.id_hotel
10 WHERE b.check_out_date >= CURRENT_DATE - 30
11     AND b.check_out_date <= CURRENT_DATE
12 GROUP BY h.name_hotel
```

Below the script editor, there are tabs for "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing the following message:

```
CREATE VIEW
Query returned successfully in 64 msec.
```

Query		Query History
1	SELECT *	
2	FROM hotel.view_hotel_income_last_month	

Data Output		Messages	Notifications
Showing rows: 1 to 2 Page No: 1 of 1			
ОТЕЛЬ	Доход за прошедший месяц		
character varying	numeric		
1	Grand Hotel	10000	
2	Nevsky Palace	8000	

Добавить новый номер 301 в отель Grand Hotel. Идентификатор отеля определить с помощью подзапроса.

Состояние таблицы до INSERT

```
SELECT
  r.id_room,
  h.name_hotel,
  r.number_room,
  r.floor,
  r.status_room
FROM hotel.room r
JOIN hotel.hotel h
  ON r.id_hotel = h.id_hotel
ORDER BY r.id_room;
```

hotel_db/postgres@PostgreSQL 18

Query Query History

```
1 SELECT
2     r.id_room,
3     h.name_hotel,
4     r.number_room,
5     r.floor,
6     r.status_room
7 FROM hotel.room r
8 JOIN hotel.hotel h
9     ON r.id_hotel = h.id_hotel
10 ORDER BY r.id_room;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	1	Grand Hotel	101	1	free
2	2	Grand Hotel	102	1	booked
3	3	Grand Hotel	201	2	occupied
4	4	Nevsky Palace	101	1	free
5	5	Nevsky Palace	202	2	booked

INSERT с подзапросом

```
INSERT INTO hotel.room(
id_room,
id_hotel,
id_type_room,
floor,
number_room,
status_room
)
VALUES(
6,
(SELECT id_hotel
FROM hotel.hotel
WHERE name_hotel = 'Grand Hotel'
),
1,
3,
301,
'free'
)
```

Query

Query History

↗

```
1  INSERT INTO hotel.room(
2  id_room,
3  id_hotel,
4  id_type_room,
5  floor,
6  number_room,
7  status_room
8  )
9  VALUES(
10  6,
11  (SELECT id_hotel
12  FROM hotel.hotel
13  WHERE name_hotel = 'Grand Hotel'
14  ),
15  1,
16  3,
17  301,
18  'free'
19  )
20
```

Data Output

Messages

Notifications

↗

INSERT 0 1

Query returned successfully in 127 msec.

В результате появился номер 301

Query

Query History

↗

```
1  SELECT
2      r.id_room,
3      h.name_hotel,
4      r.number_room,
5      r.floor,
6      r.status_room
7  FROM hotel.room r
8  JOIN hotel.hotel h
9      ON r.id_hotel = h.id_hotel
10 ORDER BY r.id_room;
```

Data Output

Messages

Notifications

↗

⌵

⌵

🗑

📄

⬇

⌵

SQL

Showing rows: 1 to 6

🔍

📄

Page No: 1

of 1

⏪

⏩

⏴

⏵

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	1	Grand Hotel	101	1	free
2	2	Grand Hotel	102	1	booked
3	3	Grand Hotel	201	2	occupied
4	4	Nevsky Palace	101	1	free
5	5	Nevsky Palace	202	2	booked
6	6	Grand Hotel	301	3	free

Изменить статус номера 301 отеля Grand Hotel на booked. Идентификатор отеля определить с помощью подзапрос

состояние до UPDATE

Query Query History

1
2
3
4
5
6
7
8
9
10
11

```
SELECT
  r.id_room,
  h.name_hotel,
  r.number_room,
  r.floor,
  r.status_room
FROM hotel.room r
JOIN hotel.hotel h
  ON r.id_hotel = h.id_hotel
WHERE r.number_room = 301
      AND h.name_hotel = 'Grand Hotel';
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 1

Page No: 1 of 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	6	Grand Hotel	301	3	free

update запрос для смены статуса с free на booked

```
UPDATE hotel.room
SET status_room = 'booked'
WHERE number_room = 301
      AND id_hotel = (
        SELECT id_hotel
        FROM hotel.hotel
        WHERE name_hotel = 'Grand Hotel'
      )
```

Query

Query History

1

2

3

4

5

6

7

8

```
UPDATE hotel.room
SET status_room = 'booked'
WHERE number_room = 301
AND id_hotel = (
    SELECT id_hotel
    FROM hotel.hotel
    WHERE name_hotel = 'Grand Hotel'
);
```

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 99 msec.

hotel_db/postgres@PostgreSQL 18

No limit

Query

Query History

```

1  SELECT
2      r.id_room,
3      h.name_hotel,
4      r.number_room,
5      r.floor,
6      r.status_room
7  FROM hotel.room r
8  JOIN hotel.hotel h
9      ON r.id_hotel = h.id_hotel
10 WHERE r.number_room = 301
11      AND h.name_hotel = 'Grand Hotel';

```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

Page No: 1 of 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	6	Grand Hotel	301	3	booked

Доказательство существования 301 номера

Query Query History

```
1 SELECT
2     r.id_room,
3     h.name_hotel,
4     r.number_room,
5     r.floor,
6     r.status_room
7 FROM hotel.room r
8 JOIN hotel.hotel h
9     ON r.id_hotel = h.id_hotel
10 WHERE r.number_room = 301
11     AND h.name_hotel = 'Grand Hotel';
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1 Page No: 1

	id_room integer	name_hotel character varying	number_room integer	floor integer	status_room character varying
1	6	Grand Hotel	301	3	booked

```
DELETE FROM hotel.room
WHERE number_room = 301
AND id_hotel = (
    SELECT id_hotel
    FROM hotel.hotel
    WHERE name_hotel = 'Grand Hotel'
)
```

Query Query History

```
1 DELETE FROM hotel.room
2 WHERE number_room = 301
3     AND id_hotel = (
4         SELECT id_hotel
5         FROM hotel.hotel
6         WHERE name_hotel = 'Grand Hotel'
7     )
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 104 msec.

После DELETE номер удален

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

SELECT

r.id_room,

h.name_hotel,

r.number_room,

r.floor,

r.status_room

FROM hotel.room r

JOIN hotel.hotel h

ON r.id_hotel = h.id_hotel

WHERE r.number_room = 301

AND h.name_hotel = 'Grand Hotel';

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗑️

📥

📥

📈

SQL

id_room	name_hotel	number_room	floor	status_room
integer	character varying	integer	integer	character varying

Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Query

Query History

1

2

3

4

5

6

7

EXPLAIN ANALYZE

SELECT

id_room,

number_room,

status_room

FROM hotel.room

WHERE status_room = 'free'

Data Output

Messages

Notifications

Showing rows: 1 to 6

Page No: 1 of 1

QUERY PLAN

text

1

2

3

4

5

6

Seq Scan on room (cost=0.00..22.75 rows=5 width=40) (actual time=0.047..0.049 rows=2.00 loop...

Filter: ((status_room)::text = 'free'::text)

Rows Removed by Filter: 3

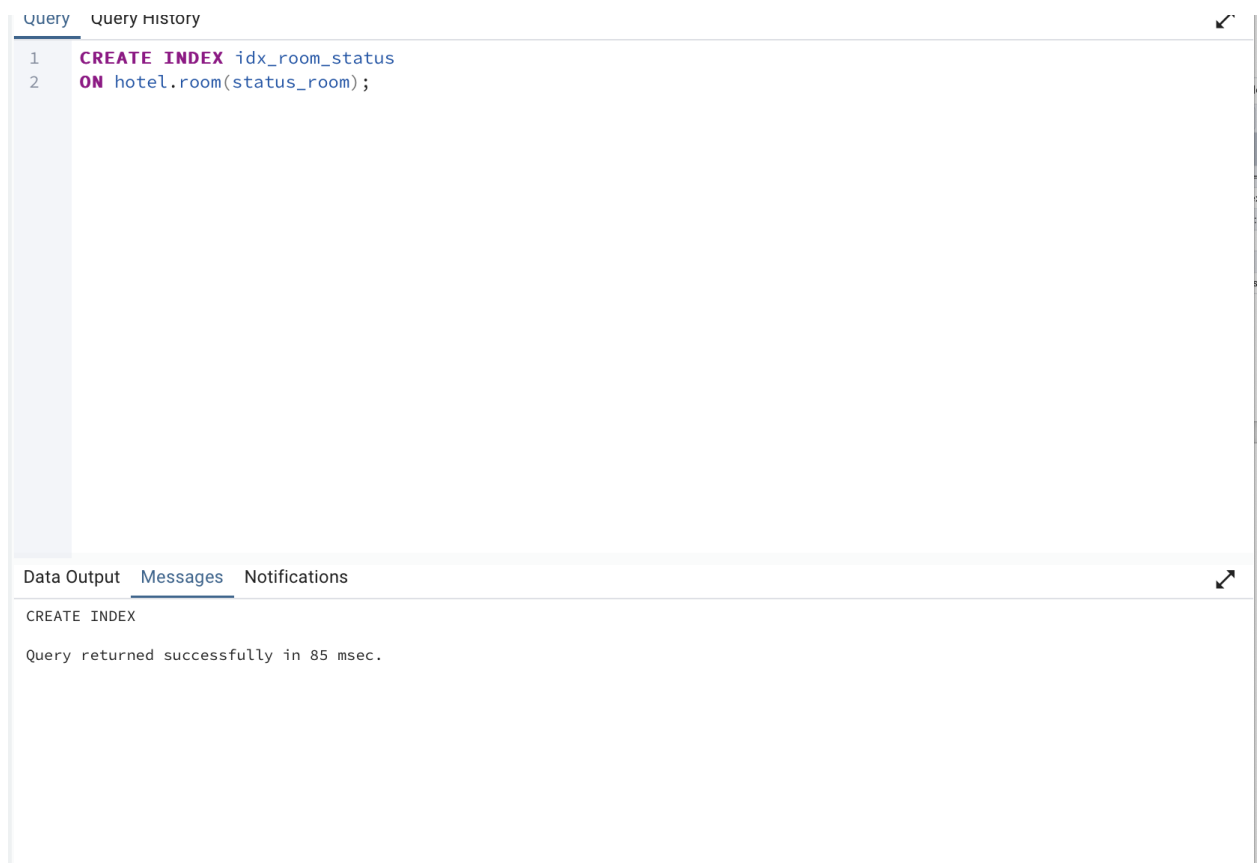
Buffers: shared hit=1

Planning Time: 0.132 ms

Execution Time: 0.063 ms

Создание индекса

CREATE INDEX idx_room_status
ON hotel.room(status_room);



После создания простого индекса по полю `status_room` время выполнения запроса уменьшилось с 0.063 ms до 0.039 ms. Однако в плане выполнения остался Seq Scan, так как таблица содержит небольшое количество строк, и PostgreSQL считает последовательное сканирование более выгодным, чем использование индекса.

Анализ плана выполнения запроса без составного индекса

Для анализа выполнения запроса была использована команда `EXPLAIN ANALYZE`, так как по методике лабораторной работы необходимо получить план запроса и сравнить время выполнения запроса без индекса и с индексом

```
EXPLAIN ANALYZE
SELECT
  id_room,
  number_room,
  floor,
  status_room
FROM hotel.room
WHERE id_hotel = 1
AND status_room = 'free'
```

Query

Query History

1

2

3

4

5

6

7

8

9

EXPLAIN ANALYZE

SELECT

id_room,

number_room,

floor,

status_room

FROM hotel.room

WHERE id_hotel = 1

AND status_room = 'free'

Data Output

Messages

Notifications

Showing rows: 1 to 8

Page No: 1 of 1

QUERY PLAN

text

1

2

3

4

5

6

7

8

Seq Scan on room (cost=0.00..1.07 rows=1 width=44) (actual time=0.027..0.028 rows=1.00 loop...

Filter: ((id_hotel = 1) AND ((status_room)::text = 'free'::text))

Rows Removed by Filter: 4

Buffers: shared hit=1

Planning:

Buffers: shared hit=6 dirtied=2

Planning Time: 0.858 ms

Execution Time: 0.057 ms

CREATE INDEX idx_room_hotel_status
ON hotel.room(id_hotel, status_room);

Query Query History

1

CREATE INDEX idx_room_hotel_status

2

ON hotel.room(id_hotel, status_room);

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 60 msec.

Проверяем время после сотавного индекса

EXPLAIN ANALYZE

SELECT

id_room,

number_room,

floor,

status_room

FROM hotel.room

WHERE id_hotel = 1

AND status_room = 'free';

Query

Query History

1

2

3

4

5

6

7

8

9

EXPLAIN ANALYZE

SELECT

id_room,

number_room,

floor,

status_room

FROM hotel.room

WHERE id_hotel = 1

AND status_room = 'free';

Data Output

Messages

Notifications

Showing rows: 1 to 8

Page No: 1 of 1

QUERY PLAN

text

1

2

3

4

5

6

7

8

Seq Scan on room (cost=0.00..1.07 rows=1 width=44) (actual time=0.027..0.028 rows=1.00 loop...

Filter: ((id_hotel = 1) AND ((status_room)::text = 'free'::text))

Rows Removed by Filter: 4

Buffers: shared hit=1

Planning:

Buffers: shared hit=19 read=1

Planning Time: 1.368 ms

Execution Time: 0.057 ms

После создания составного индекса заметного ускорения не произошло:
время выполнения осталось тем же — **0.057 ms**.

Это связано с тем, что таблица room содержит очень небольшое количество строк, поэтому PostgreSQL считает более выгодным выполнить **последовательное сканирование (Seq Scan)**, чем использовать индекс.

Таким образом, составной индекс был успешно создан, однако на данной небольшой учебной таблице его влияние на производительность практически не проявилось.